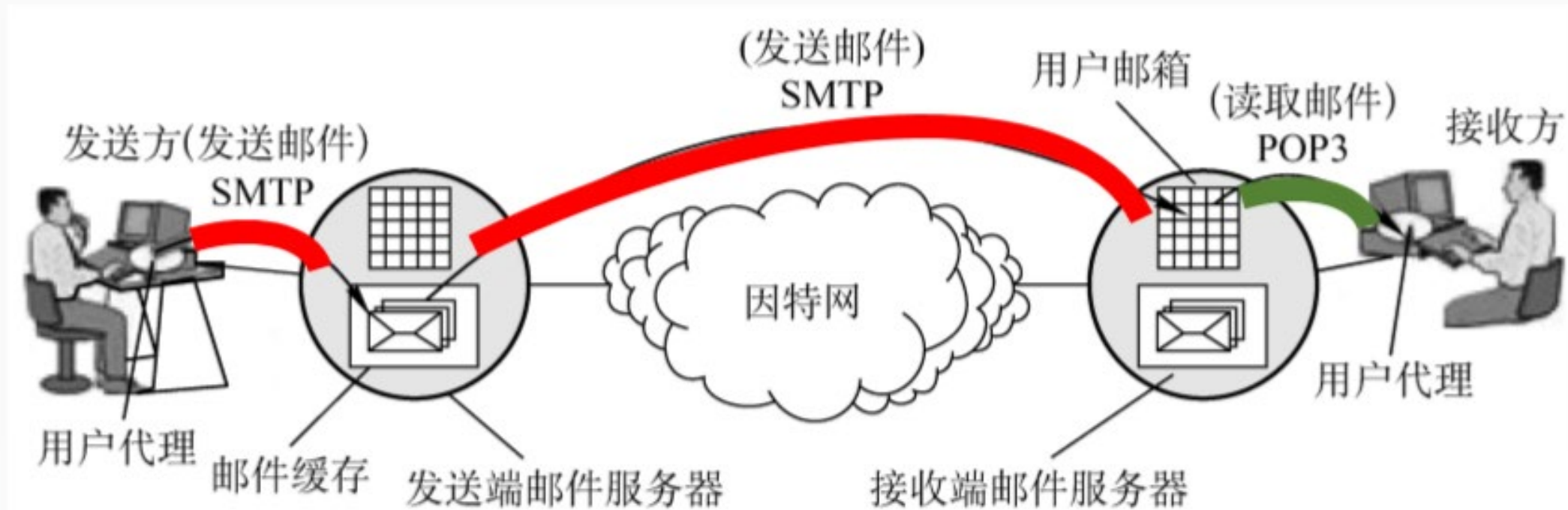


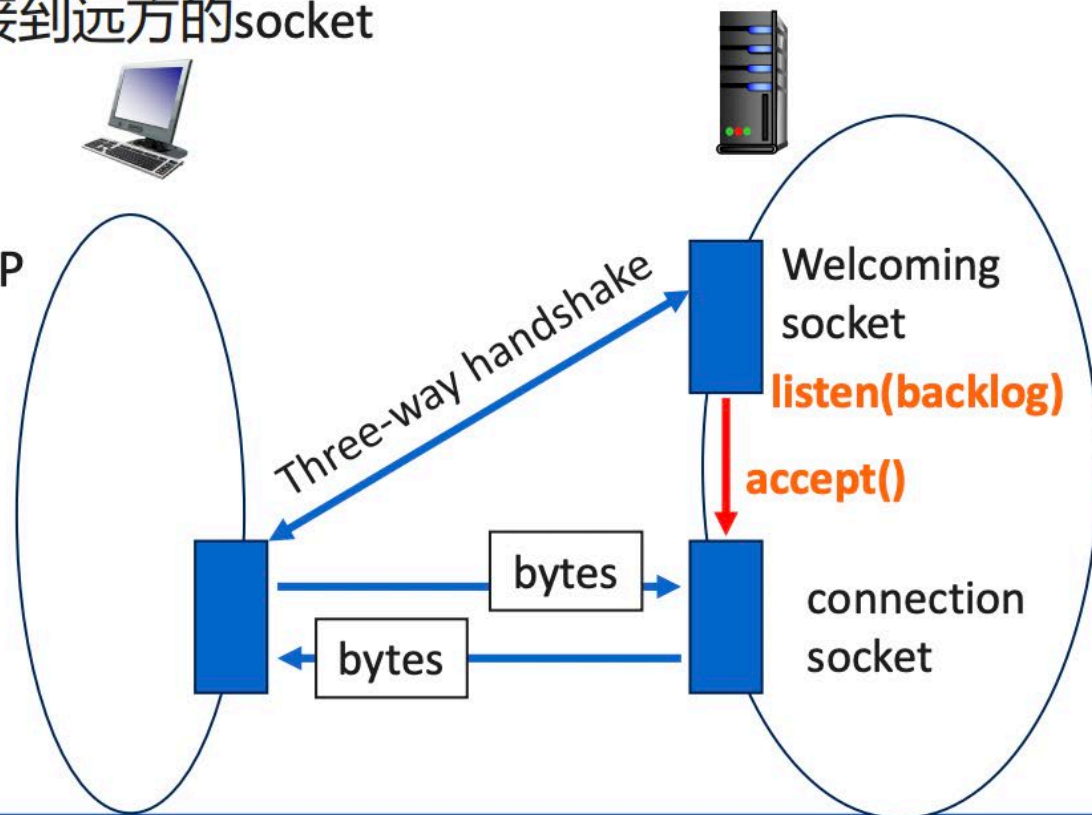
SOCKET编程实现SMTP客户端

电子邮件系统结构



TCP socket 编程

- 面向连接的可靠的、在序的、字节流的数据传输服务
- 服务方在某个TCP socket处调用listen(backlog), 等待客户方建立连接:
 - 服务方调用accept(), 在通过三次握手过程完成连接建立后, 创建一个新的socket, 负责之后与该client的数据传输
 - backlog: 等待accept的连接的最大个数
- 客户方创建一个TCP socket, 调用connect((host, port)连接到远方的socket
- 不保护用户数据的边界, 看成字节流
 - send和recv时需要检查实际发送和接收的字节数
 - sendall(data): 保证传递的数据都已递交给发送方TCP



TCP Socket编程

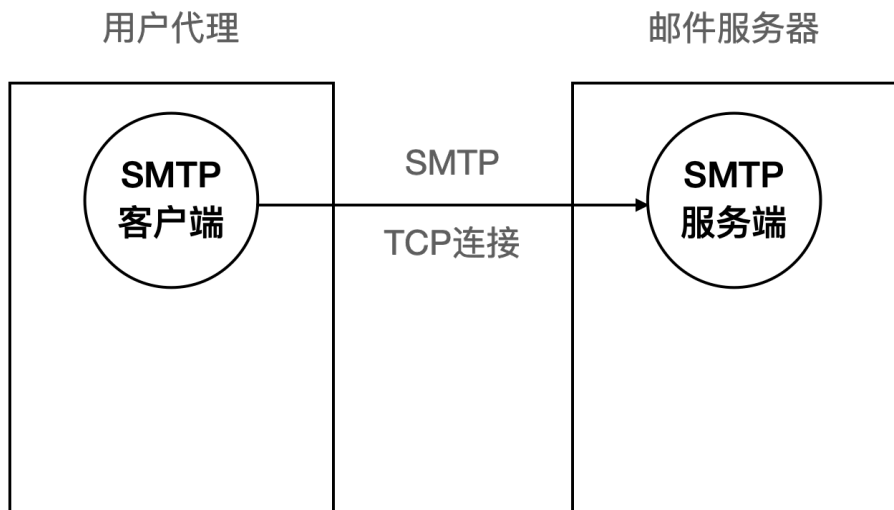
tcp-client.py

```
import socket
server = 'localhost'
port = 12000
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((server, port))
message = input('Input lowercase sentence:')
client_socket.send(message.encode())
modified_message = client_socket.recv(1024)
print('From Server: ', modified_message.decode())
client_socket.close()
```

tcp-server.py

```
import socket
server_port = 12000
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('', server_port))
server_socket.listen(1)
print('The server is ready to receive at', server_socket.getsockname())
while True:
    connection_socket, client_address = server_socket.accept()
    print('Connection from', client_address)
    message = connection_socket.recv(1024)
    modified_message = message.decode().upper()
    connection_socket.send(modified_message.encode())
    connection_socket.close()
```


客户端功能



功能：

撰写

发送—> socket + smtp

收件—> socket + pop3

基本功能：

1 编辑邮件

2 发送邮件，支持单发、群发（socket+smtp）

可选功能：

1 草稿箱

2 已发送

3 通讯录

.....

说明：

1 程序最核心的功能是“发送邮件”，在保证该功能完善的基础上，如果时间充分，可酌情考虑实现一些可选功能；

2 字符界面/图形界面都可以；

3 不允许使用smtpplib以及其他类似的python第三方库，要保证能体现出client和server之间的socket通信过程以及客户端基于socket通信发送的一些smtp命令和收到的服务器的响应，参考示例程序；

4 邮件服务器可使用腾讯的邮件服务器等

服务器域名：smtp.qq.com

端口：587

腾讯的邮件服务器默认是关闭SMTP服务的，打开SMTP服务的步骤为：登录个人qq邮箱->设置->账户->开启POP3/SMTP服务

SMTP服务开启后，会获得一个授权码，该授权码即为登录腾讯邮件服务器的密码，登录邮件服务器的账号为自己的qq邮箱账号



示例程序——错误程序

```
import smtplib
from email.mime.text import MIMEText
from email.header import Header

# 第三方 SMTP 服务
mail_host = "smtp.qq.com" # 设置服务器
mail_user = "xxx@qq.com" # 用户名
mail_pass = "xxx" # 口令

sender = 'xxx@qq.com'
receivers = ['xxx@qq.com', 'yyy@qq.com'] # 接收邮件，可设置为你的QQ邮箱或者其他邮箱

message = MIMEText('Python 邮件发送测试...', 'plain', 'utf-8')
message['From'] = Header("xxx", 'utf-8')
message['To'] = Header("yyy", 'utf-8')

subject = 'Python SMTP 邮件测试'
message['Subject'] = Header(subject, 'utf-8')

try:
    smtpObj = smtplib.SMTP()
    smtpObj.connect(mail_host, 587)
    smtpObj.login(mail_user, mail_pass)
    smtpObj.sendmail(sender, receivers, message.as_string())
    print("邮件发送成功")
except smtplib.SMTPException as e:
    print("Error: 无法发送邮件")
```

示例程序——正确程序

```
from socket import *
from email.base64mime import body_encode

msg = "\r\n I love computer networks!"
endMsg = "\r\n.\r\n"
# 选择一个邮件服务
mailServer = "smtp.qq.com"
# 发送方地址和接收方地址, from 和 to
fromAddress = "xxx@qq.com"
toAddress = "yyy@qq.com"
# 发送方, 验证信息, 由于邮箱输入信息会使用base64编码, 因此需要进行编码
username = "xxx@qq.com" # 输入自己的用户名对应的编码
password = "xxx" # 此处不是自己的密码, 而是开启SMTP服务时对应的授权码

# 创建客户端套接字并建立连接
serverPort = 587 # SMTP使用587号端口
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((mailServer, serverPort)) # connect只能接收一个参数
# 从客户套接字中接收信息
recv = clientSocket.recv(1024).decode()
print(recv)
if '220' != recv[:3]:
    print('220 reply not received from server.')

# 发送 HELO 命令并且打印服务端回复
heloCommand = 'HELO Alice\r\n'
clientSocket.send(heloCommand.encode()) # 随时注意对信息编码和解码
recv1 = clientSocket.recv(1024).decode()
print(recv1)
```

```
if '250' != recv1[:3]:
    print('250 reply not received from server.')
# 发送 HELO 命令并且打印服务端回复
# 开始与服务器的交互, 服务器将返回状态码250,说明请求动作正确完成
heloCommand = 'HELO MyName\r\n'
clientSocket.send(heloCommand.encode()) # 随时注意对信息编码和解码
recv1 = clientSocket.recv(1024).decode()
print(recv1)
if '250' != recv1[:3]:
    print('250 reply not received from server.')

# 发送"AUTH PLAIN"命令, 验证身份.服务器将返回状态码334 (服务器等待用户输入验证信息)
user_pass_encode64 = body_encode(f"\0{username}\0{password}".encode('ascii'), eol="")
clientSocket.sendall(f'AUTH PLAIN {user_pass_encode64}\r\n'.encode())
recv2 = clientSocket.recv(1024).decode()
print(recv2)

.....
```